

Mise en place d'une suite de tests automatisés

Projet SentimentIA — API d'analyse de sentiments

Rapport d'atelier

Tests unitaires, tests d'intégration, intégration continue, TDD, tests de performance et audit de sécurité

1. Introduction et contexte

Dans le cadre de cet atelier, j'ai pris en charge la mise en place de la suite de tests complète de SentimentIA, une API d'analyse de sentiments destinée à traiter les avis clients. Le travail s'est déroulé en cinq étapes successives : la mise en place de l'environnement (TP0), l'écriture des tests unitaires (TP1), les tests d'intégration de l'API (TP2), la construction d'un pipeline d'intégration continue (TP3), puis un cycle TDD accompagné de tests de performance et d'un audit de sécurité (TP4).

L'objectif n'était pas de développer l'application elle-même, fournie au départ, mais d'en garantir la fiabilité par une stratégie de test progressive. J'ai volontairement structuré ma démarche pour que chaque niveau de test réponde à une question précise : la logique métier est-elle correcte, l'API répond-elle bien, le code reste-t-il sain, et tient-il la charge ?

Au fil de l'atelier, quatre défauts ont été identifiés puis corrigés grâce aux tests. Je les détaille dans les sections correspondantes et les récapitule en conclusion.

2. Présentation de l'application

SentimentIA expose une API HTTP construite avec FastAPI. Le cœur métier, la classe `SentimentModel`, analyse un texte en comptant les mots positifs et négatifs qu'il contient, puis en déduit un label (POSITIVE, NEGATIVE ou NEUTRAL) et un score de confiance compris entre 0 et 1. La validation des données entrantes est assurée par Pydantic : un texte vide, trop long ou absent est automatiquement rejeté.

L'API met à disposition quatre points d'entrée :

Méthode	URL	Rôle
GET	/health	Vérifier que l'API est opérationnelle
POST	/predict	Analyser un texte et renvoyer le sentiment
GET	/stats	Consulter les compteurs de prédictions
POST	/reset	Remettre les compteurs à zéro

Les compteurs sont maintenus en mémoire. L'endpoint `/reset` joue un rôle important côté tests : il permet de repartir d'un état propre avant chaque vérification, ce qui garantit l'indépendance des tests entre eux.

3. Mise en place de l'environnement (TP0)

J'ai d'abord créé l'arborescence du projet en séparant le code source (`src/`) des tests (`tests/`). Les fichiers `__init__.py` rendent ces dossiers importables comme des packages Python, ce qui évite les erreurs `ModuleNotFoundError` au moment de l'exécution des tests.

J'ai ensuite isolé les dépendances dans un environnement virtuel, puis installé les bibliothèques en versions épinglées dans `requirements.txt`. Épingler les versions est une habitude que je tiens à conserver : cela garantit que mon poste, celui d'un collègue et le serveur d'intégration travaillent tous avec exactement le même environnement.

La configuration de pytest est centralisée dans `pytest.ini`, qui indique notamment de ne chercher les tests que dans le dossier `tests/`. Pour valider l'installation, deux smoke tests étaient fournis : ils vérifient simplement que l'API démarre et répond. Les deux passent, ce qui confirme que l'environnement est fonctionnel.

Ce que j'ai retenu des questions du TP0

En relisant la validation Pydantic, j'ai vérifié qu'un texte vide, un texte de plus de 5000 caractères ou un champ absent produisent tous une erreur 422 (Unprocessable Entity), renvoyée avant même que la logique métier soit atteinte. Fait intéressant : lorsque le modèle lève une `SentimentError` (texte sans aucun mot analysable), l'API renvoie elle aussi un 422, mais cette fois depuis la logique métier et non depuis la validation du schéma. Le code HTTP est identique, l'origine diffère.

4. Tests unitaires du modèle (TP1)

Les tests unitaires ciblent la méthode `predict()` en isolation, sans passer par l'API. J'ai d'abord introduit une fixture `model` qui fournit une instance fraîche du modèle à chaque test : cela évite la répétition et, surtout, si le constructeur de la classe évolue, je n'ai qu'un seul endroit à modifier.

J'ai couvert les trois labels possibles, le cas de l'exception, et utilisé la paramétrisation pour vérifier plusieurs textes en une seule fonction. Un même bloc `@parametrize` avec cinq jeux de données génère cinq tests distincts, chacun identifiable individuellement dans le rapport. J'ai également écrit un test avec un mock (`patch.object`) qui vérifie que `predict()` est appelée avec le bon argument, sans exécuter sa vraie logique. Ce mécanisme deviendrait indispensable si la méthode appelait un jour une API externe payante : on testerait alors le contrat d'appel sans déclencher d'appel réseau réel.

Premier défaut détecté : le score non borné

Un de mes tests envoyait un texte contenant les dix mots positifs du modèle et vérifiait que le score ne dépassait pas 1.0. Il a échoué en affichant un score de 1.6. La formule $0.6 + 0.1 \times n$ n'était pas plafonnée. J'ai corrigé le calcul en l'encadrant par `min(..., 1.0)`. Ce type de défaut, une formule jamais testée à ses valeurs limites, est typique des bugs qui passent inaperçus en conditions normales.

Après correction, la couverture de `src/model.py` atteint 100 % : toutes les branches (positif, négatif, neutre, exception, plafonnement) sont exercées.

5. Tests d'intégration de l'API (TP2)

Les tests d'intégration vérifient le comportement de bout en bout : une requête entre par HTTP, traverse FastAPI, Pydantic et le modèle, puis ressort en JSON. J'ai regroupé les outils communs dans `tests/conftest.py`, chargé automatiquement par pytest. On y trouve un client HTTP réutilisable, des générateurs de textes réalistes basés sur Faker, et une fixture `reset_stats`.

Cette fixture utilise `yield` plutôt que `return` afin de séparer ce qui s'exécute avant le test de ce qui s'exécuterait après. La remise à zéro est placée avant le `yield` pour que chaque test démarre d'un état propre. Surtout, elle est déclarée en `autouse=True` : elle s'applique alors à tous les tests sans que j'aie à la mentionner, ce qui garantit leur isolation systématique.

J'ai couvert les quatre endpoints ainsi que les quatre cas d'erreur Pydantic (texte vide, trop long, manquant, mauvais type), tous attendus en 422. En inspectant une réponse 422, j'ai constaté que FastAPI renvoie une structure de la forme {"detail": [...]}, où le message lisible se trouve dans detail[0]["msg"].

Deuxième défaut détecté : l'état partagé

L'atelier proposait deux tests volontairement dépendants l'un de l'autre. En les lançant dans un ordre puis dans l'autre, j'ai observé qu'ils ne donnaient pas le même résultat : l'un suppose que les compteurs partent de zéro, ce qui n'est vrai que si l'autre ne s'est pas exécuté avant. Comme l'ordre d'exécution de pytest n'est pas garanti, ce genre de dépendance produit des échecs imprévisibles. La fixture autouse `reset_stats` résout le problème en rendant chaque test indépendant.

J'ai aussi généré un rapport au format JUnit XML (`report.xml`). Chaque balise `testcase` porte un attribut `time` indiquant sa durée. Ce format structuré est nécessaire au serveur d'intégration, qui le parse de façon fiable, marque le build en succès ou en échec, et archive l'historique des exécutions pour suivre les tendances.

6. Pipeline d'intégration continue (TP3)

J'ai conteneurisé l'application avec un Dockerfile basé sur `python:3.12-slim`, dont la construction aboutit sans erreur. Un fichier `docker-compose` orchestre deux services : l'API et un serveur Jenkins. Le pipeline lui-même est décrit dans un `Jenkinsfile` composé de six étapes enchaînées :

- Checkout : récupération du code source
- Install : installation des dépendances
- Lint : analyse statique Pylint, avec un seuil minimal fixé à 7,0
- Tests Unitaires : pytest avec couverture et rapport JUnit
- Tests Integration : pytest sur l'API avec rapport JUnit
- Security Scan : audit de sécurité Bandit

Si une étape échoue, le build est marqué en échec. J'ai mesuré la qualité du code source localement avant intégration :

Outil	Résultat obtenu	Seuil exigé
Pylint	9,61 / 10	7,0 minimum
Bandit	0 alerte (aucune Low/Medium/High)	aucune alerte critique

Concernant la sécurité, le code source de `SentimentIA` ne présente aucune vulnérabilité détectable par Bandit : pas de secret en dur, pas d'exécution dynamique de code, pas d'écoute réseau ouverte côté Python. L'écoute sur `0.0.0.0`, nécessaire au fonctionnement dans un conteneur, figure uniquement dans le Dockerfile, hors du périmètre analysé. Le stage de sécurité reste donc vert.

7. TDD, performance et sécurité (TP4)

7.1 Cycle TDD : la gestion des négations

Le modèle initial classait « pas bien » comme positif, puisqu'il repérait le mot « bien » sans tenir compte de la négation qui le précède. J'ai traité ce point en suivant rigoureusement le cycle TDD.

J'ai d'abord écrit les tests décrivant le comportement attendu, puis je les ai exécutés pour confirmer qu'ils échouaient (phase RED). Cette étape est essentielle : un test qui échoue avant l'implémentation prouve qu'il teste réellement la nouvelle fonctionnalité. À l'inverse, un test qui passerait sans que le code existe serait un test trompeur, dont les assertions sont probablement fausses.

J'ai ensuite implémenté la détection des négations dans `predict()` : le texte est parcouru mot à mot, et un drapeau indique si le mot-clé suivant doit être inversé. Les mots de négation reconnus sont pas, plus, jamais, sans et aucun. Une fois le code écrit, les tests sont passés au vert (phase GREEN). Enfin, lors du refactor, j'ai sorti la liste des négations en attribut de classe, complété la docstring et vérifié que le score restait correctement plafonné.

Aucune régression n'est apparue : les 27 tests de `test_model.py` passent (18 issus du TP1 et 9 liés aux négations), et la couverture de `src/model.py` reste à 100 %. C'est logique : chaque test écrit en amont du code force l'exécution de lignes nouvelles, qui deviennent couvertes dès leur implémentation.

7.2 Tests de performance avec Locust

J'ai écrit un scénario Locust simulant des utilisateurs qui sollicitent l'API. J'ai attribué un poids de 3 à `/predict` et de 1 à `/stats` : le poids représente la fréquence relative d'une action, et ce ratio reflète un usage réaliste où l'on analyse des textes bien plus souvent qu'on ne consulte les statistiques. J'ai exécuté deux campagnes, à 50 puis 100 utilisateurs virtuels.

Charge	Requêtes	Latence P50	Latence P95	Échecs
50 utilisateurs	2 660	4 ms	7 ms	0
100 utilisateurs	4 993	4 ms	14 ms	0

Le seuil que je m'étais fixé pour `/predict` était un P95 inférieur à 500 ms. Il est largement respecté dans les deux cas. En doublant la charge, la latence P95 double (de 7 à 14 ms) mais aucune erreur n'apparaît : l'API évolue de façon linéaire sur cette plage et dispose d'une marge confortable avant saturation.

7.3 Audit de sécurité après le TDD

J'ai relancé Bandit après les modifications du cycle TDD pour m'assurer que l'ajout de la logique de négation n'avait introduit aucune faille. L'audit ne relève aucune alerte. Le code source reste sain et le pipeline conserve ses six étapes au vert.

8. Conclusion

Au terme de l'atelier, le projet dispose d'une suite de 42 tests, tous au vert, et d'une couverture globale de 98 % (100 % sur la logique métier). L'application est conteneurisée, passe les audits de qualité et de sécurité, et tient la charge avec une latence très faible. L'ensemble s'intègre dans un pipeline automatisé reproductible.

J'ai surtout retenu que les tests ne servent pas seulement à confirmer ce qui fonctionne, mais qu'ils révèlent activement les défauts. Quatre d'entre eux ont été identifiés et corrigés au fil de l'atelier :

Étape	Défaut	Comment je l'ai détecté	Correction apportée
TP1	Score dépassant 1.0	Test vérifiant score <= 1.0	Plafonnement par min(..., 1.0)
TP2	Tests dépendants entre eux	Inversion de l'ordre d'exécution	Fixture reset automatique (autouse)
TP3	Qualité / sécurité du code	Pylint et Bandit dans le pipeline	Code maintenu propre (9,61/10, 0 alerte)
TP4	Négations ignorées	Test TDD en phase RED	Parcours des tokens avec drapeau de négation

Cette progression, du test unitaire le plus fin jusqu'au pipeline complet, m'a permis de comprendre concrètement comment une équipe garantit la fiabilité d'un logiciel avant de le livrer, et pourquoi chaque test doit rester indépendant, lisible et reproductible.